Sparse Autoencoders Learn Graded Latent Activations for Relative Magnitude-based Composition

Anonymous Authors[1]

Abstract

Transformers can learn a mechanism that represents multiple features using a single set of basis vectors and differentiates them using relative magnitudes. The impact on sparse autoencoders (SAEs) has not been studied, but this may have significant implications for interpretability using them. In this paper, we find that SAEs reliably learn graded latent activations which represent different list orderings at different magnitudes. We additionally find that we can steer the transformer using these latents to change the order of model outputs. In future work, we will search for graded latent activations in popular pretrained SAEs such as Gemma Scope. Anonymised code is available here: https://anonymous.4open.science/r/graded-latents-70FA.

1. Introduction

Sparse Autoencoders (SAEs) (Bricken et al., 2023; Huben et al., 2024) and their variants (Gao et al., 2025; Bussmann et al., 2024; Rajamanoharan et al., 2024b; Leask et al., 2025b; Costa et al., 2025; Bussmann et al., 2025) are popular tools for recovering features from dense internal activations in language models. Recent work has examined whether SAEs produce robust dictionaries. For example, Chanin et al. (2026) find that using sparsity as a proxy can cause feature absorption and Leask et al. (2025a) find that SAEs do not find a unique and complete dictionary of atomic features. However, Farrell et al. (2025) suggest that even if an SAE recovers the correct dictionary, it may not provide a complete understanding of the model. They identify a novel 'relative magnitude-based' feature composition mechanism in a toy attention-only transformer, and predict that an SAE will learn graded latent activations that are not necessarily independently interpretable. Since interpretability with SAEs commonly assumes this independence for transformer features (Elhage et al., 2022; Bricken et al., 2023), graded latent activations may present a problem.

In this paper, we verify this prediction across a suite of BatchTopK (Bussmann et al., 2024), JumpReLU (Rajamanoharan et al., 2024b) and Matryoshka (Bussmann et al., 2025) SAEs and find that they reliably learn latents with graded activations corresponding to different bigram orderings. We aim to set the stage for further analysis on real-world pretrained SAEs such as the Gemma Scope releases (Lieberum et al., 2024; McDougall et al., 2025) which use JumpReLU and Matryoshka SAEs.

Our paper is structured as follows. Section 3 outlines our SAE training and evaluation methodology. In Section 4, we analyse a specific BatchTopK SAE and find two latents with graded activations that can be steered to swap the model output from (x, y) to (y, x). They can be identified using their activation magnitudes which correlate with the difference between transformer attention weights. We check for generalisation to other SAE configurations, including JumpReLU and Matryoshka SAEs, using an automated pipeline that, given an SAE, identifies latents of interest using correlations as described, and steers them to swap outputs. Across several SAEs, we find we can reliably swap between 25-70% of the transformer's original outputs by steering a single SAE latent in this way.

Our paper is intended to provide a foundation for future work searching for graded latent activations in pretrained SAEs for LLMs. Searching for these systematically in large SAEs seems difficult. By providing avenues for automatic detection, we aim to foster this work.

2. Related Work

Sparse Autoencoders. Models can represent more than n features in n-dimensional activations through superposition, which complicates interpretability (Elhage et al., 2022). Sparse autoencoders (SAEs) (Bricken et al., 2023; Huben et al., 2024) and their variants (Gao et al., 2025; Bussmann et al., 2024; Rajamanoharan et al., 2024b; Bussmann et al., 2025; Karvonen et al., 2025; Leask et al., 2025b) decompose dense activations into sparse feature representations in

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

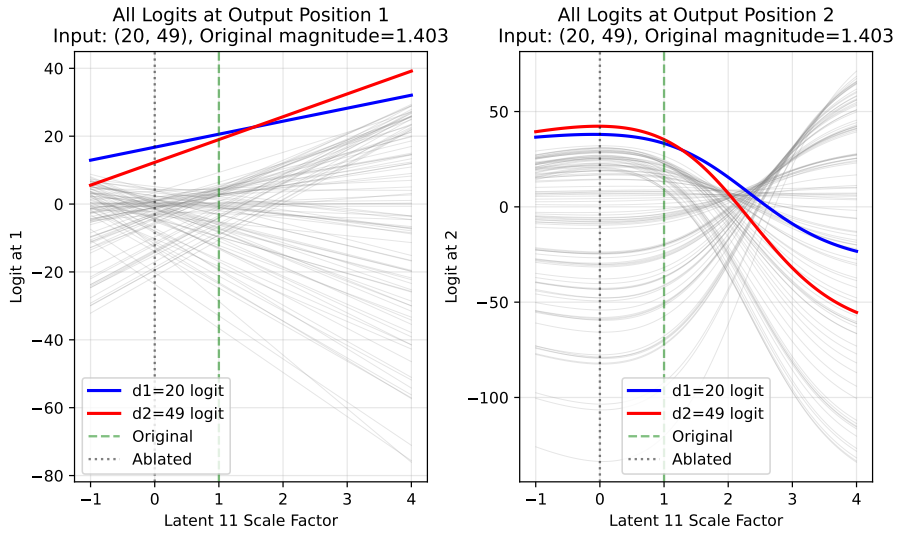

Figure 1. This plot shows the logits at output positions o_1 (left) and o_2 (right) for input $(20, 49)$ as latent 11's activation is scaled by factor p (x-axes). Each line shows the logit for one symbol; blue = d_1, red = d_2 and all others are grey. A valid swap requires the red line to lead at o_1 and the blue line to lead at o_2 simultaneously. Here, this holds for $p \in [0.16, 2.20]$.

an attempt to address this.

In this paper, we focus on BatchTopK SAEs (Bussmann et al., 2024) since they are simple to analyse. We check for generalisation of our results in JumpReLU (Rajamanoharan et al., 2024b) and Matryoshka (Bussmann et al., 2024) SAEs both to improve the validity of our findings and support further work on Gemma Scope 1 (Lieberum et al., 2024) and 2 (McDougall et al., 2025) which use these variants.

SAEs have been successfully used in the past (Marks et al., 2025; Movva et al., 2025) but there are uncertainties over whether they learn robust dictionaries (Chanin et al., 2026; Leask et al., 2025a). In our paper, we show that even a correct dictionary may be insufficient for complete model understanding using SAEs.

Relative Magnitude-based Composition. (Farrell et al., 2025) train a two-layer attention-only transformer on an ordered bigram copying task, in which the model must encode bigram (d_1, d_2) into a single separator token, s, and reconstruct the bigram as output, (o_1, o_2) where $(o_1, o_2) = (d_1, d_2)$. They find that the model represents the bigram at s using the relative magnitude of scalar attention weights, rather than representing each possible ordering as a different direction in activation space (which other composition methods do (Wattenberg & Viégas, 2024)). Specifically,

$$\boldsymbol{r}_s^{L_1} = \alpha_{s\to d_1}(\boldsymbol{E}_{d_1}+\boldsymbol{P}_{d_1})+\alpha_{s\to d_2}(\boldsymbol{E}_{d_2}+\boldsymbol{P}_{d_2})+\boldsymbol{E}_s+\boldsymbol{P}_s$$

where $\alpha_{s\to d_i}$ is the attention weight in the first layer for query s (separator token) and key d_i (element of input bigram), and $\boldsymbol{E}_x$ and $\boldsymbol{P}_x$ are the token and positional embeddings for the token at x respectively. They predict that an SAE trained on $\boldsymbol{r}_s^{L_1}$ will learn latents corresponding to $(\boldsymbol{E}_a+\boldsymbol{P}_{d_1})$ and $(\boldsymbol{E}_b+\boldsymbol{P}_{d_2})$ with activations equal to the attention probabilities α, and that this results in graded latent activations. In our paper, we verify this behaviour across various SAE configurations.

Steering. Steering vectors are directions in the residual stream that can be linearly scaled and patched downstream to predictably alter model behaviour (Ferrando et al., 2024). For example, Arditi et al. (2024) identify a single refusal direction via difference-in-means and use it to suppress or amplify refusal in safety-tuned LMs. We apply this principle to SAE latent activations, patching the altered SAE reconstructions rather than a raw residual stream direction.

3. Methodology

Farrell et al. (2025) demonstrate the relative magnitude-based composition at the separator's residual activation after the first layer. We refer to this vector as $\boldsymbol{r}_s^{L_1}$, where s is the separator token position, and train SAEs on this. We consider BatchTopK (Bussmann et al., 2024), JumpReLU (Rajamanoharan et al., 2024b) and Matryoshka (Bussmann et al., 2025) SAEs with implementations from Marks et al. (2024). We run a grid-search over $k \in [1, 5]$ and SAE expansion factors $i \in [2, 5]$, such that $d_{\text{sae}} = i \times d_{\text{model}}$ (Table 4). We evaluate them on sparsity (L_0) and loss recovered, in line with previous work (Karvonen et al., 2025; Ferrando et al., 2024).

After a hyperparameter grid search (Table 4) we find that BatchTopK SAEs with $k \in [3, 5]$ all achieve similar top performance so we take $k = 3$ for increased sparsity. We select $d_{\text{sae}} = 128$ which is the smallest dimensionality that

Table 1. Selected SAE configuration. The optimiser, betas, schedule and auxiliary loss are fixed in the implementation (Marks et al., 2024), and so are excluded here for brevity.

Parameter	Value
SAE type	BatchTopK
Input dimension (d_{model})	64
Dictionary size (d_{SAE})	128
k	3
Actual sparsity (L_0)	3.00
Learning rate	3×10^{-4}
Batch size	4096
Training steps	50,000
Seed	1
Hardware	NVIDIA GeForce RTX 2080 Ti

achieves top average performance. We select the checkpoint with fewest dead latents (3.1%) out of BatchTopK SAEs with $k = 3, d_{\text{SAE}} = 128$ and loss recovered over 0.999. Table 1 shows the final configuration.

For analysis, we begin by inspecting max activating examples for each latent. To test if a latent has graded activations, we use steer by linearly scaling the SAE latent's activation for a given input, patch the SAE reconstruction back into the model downstream and compare the new output to the original output.

We test generalisation of our results on 1630 different SAE configurations and seeds (Table 3) where feasible. Otherwise, we use cherry-picked SAE checkpoints (Table 5) with strong performance and varying d_{SAE} and L_0 to provide a diverse test sample, although we note that this small set is insufficient for statistical significance.

4. Results

4.1. Analysing Features

We plot which the activating examples for each latent on a heatmap over all inputs (Figure 2). We identify two latent types, excluding 8% of latents which activate on fewer than 0.1% of inputs: k-symbol detectors and special latents.

k-symbol detectors (91% of latents): 88% of latents are 1-symbol detectors: they activate only if one specific symbol is present in the input at either d_1 or d_2, shown by the 'plus-sign' pattern in Figure 2. They activate most strongly where $d_1 = d_2$ (the intersection), and otherwise their activation magnitude varies only with symbol presence, not position. 3% of latents detect multiple symbols ($k > 1$) and sometimes activate with different magnitudes for different symbols (e.g., latent 5 activates stronger for symbol 7 than symbol 58), suggesting magnitude carries information.

Special latents (2 latents, 9% of activations): Only two latents (11 and 56) do not follow the above pattern. Latent 11 activates on 64% of inputs and latent 56 on 9.9%, compared to fewer than 4.5% for all other latents (Appendix C). Because they activate across most inputs, their activation cannot indicate whether a specific symbol is present, hence they are put into a 'special' category.

Moreover, Farrell et al. (2025) find that $\boldsymbol{r}_s^{L_1}$ is a linear combination of input embeddings with attention weights as coefficients, so we compare correlations between activation magnitude and $\Delta\alpha = \alpha_{s\to d_1} - \alpha_{s\to d_2}$, as shown in Appendix C, Figure 4. Latent 11 positively correlates with $\Delta\alpha$ (Pearson correlation coefficient $r = 0.807$), and latent 56 negatively correlates with $\Delta\alpha$ ($r = -0.3213$). We compute correlations over the whole dataset, and all other latents have correlation $|r| < 0.004$ with $\Delta\alpha$, suggesting that these two features are indeed special relative to the others. Latent 11 has a much stronger correlation than 56, so we refer to it as the primary special feature.

4.2. Steering Experiments

We hypothesise that special latents have graded activations. Specifically, we hypothesise that they have activation ranges corresponding to different bigram orderings. To test this hypothesis, we use steering vectors (Ferrando et al., 2024; Arditi et al., 2024) to perform a causal linear intervention: we scale special latent activations and observe whether the model's outputs change predictably.

We: **1.** Run the transformer on bigram (d_1, d_2) and record the special latent's activation magnitude m from the SAE trained on $\boldsymbol{r}_s^{L_1}$. **2.** Scale m by factor p, reconstruct $\hat{\boldsymbol{r}}_s^{L_1}$, and compute logits at (o_1, o_2). **3.** Repeat across all $p \in [0, 20]$ (step 0.05) and all input bigrams. Systematic output logit shifts across this sweep would imply that the special latent is graded.

Steering latent 11 produces output swaps for 5,002/10,000 bigrams (50.0%) and an example is shown in Figure 1. The remaining 50% are not swapped: 3,600 inputs (36%) where latent 11 does not activate ($m = 0$), and 1,424 inputs (14.2%) where it activates but no scaling produces a swap. The latter fail for the following reasons (see Appendix D for visualisations):

Negative scaling required (9.2%): Swapping o_1 requires negative p, which we reject as BatchTopK activations are non-negative by construction, so negative scaling introduces an out-of-distribution anti-feature rather than representing learned graded behaviour.

Mutually exclusive swap ranges (2.3%): Each output position can be individually swapped but not simultaneously.

No swap in observed range (1.8%): One logit remains

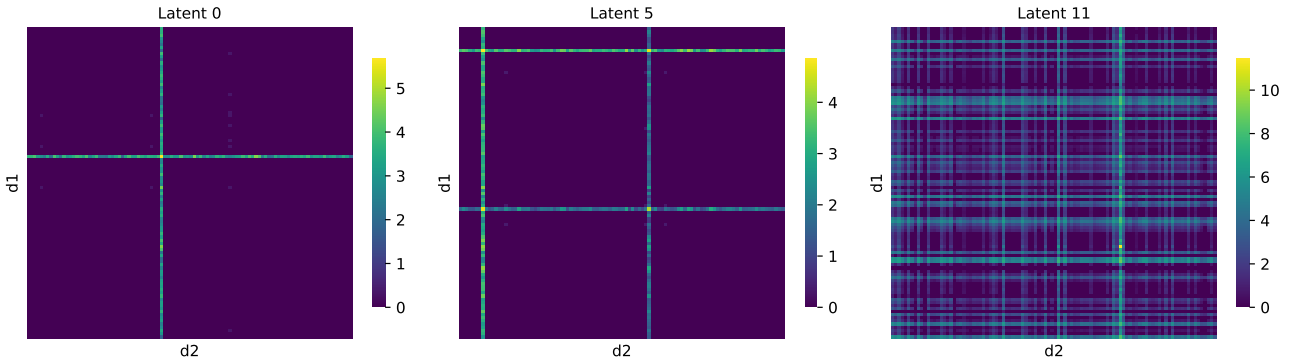

Figure 2. These 100×100 heatmaps show example SAE latent activations over all input bigrams (d_1, d_2) with $d_1, d_2 \in [0, 99]$: top-left cell is $(0, 0)$, bottom-right cell is $(99, 99)$. **Left** (1-symbol detector): latent 0 activates strongly if d_1 or $d_2 = 41$ and peaks at $d_1 = d_2 = 41$, producing a plus-sign pattern. **Centre** (k-symbol detector, $k > 1$): latent 5 activates strongly if d_1 or $d_2 \in \{7, 58\}$, producing two plus-sign patterns. **Right** (special latent): latent 11 activates on most inputs with generally much greater magnitudes than any other latent.

dominant across all $p \in [0, 20]$.

Matching bigram elements (0.8%): $d_1 = d_2$, making the swap degenerate.

For inputs that are not swapped, it is sometimes possible to scale a co-activating (non-special) latent to successfully swap outputs (see example in Appendix D). We additionally steer latent 56 and successfully swap 2.0% of outputs, which is part of the 36% of inputs that do not activate latent 11 since the two special latents never coactivate. Although we don't steer on all latents due to computational constraints, in Appendix D we attempt it on several 'non-special' latents to confirm the swapping behaviour is unique to latents 11 and 56.

Moreover, Figure 1 suggests that logits at o_1 scale linearly with p. We confirm this by fitting a linear regression model to each logit for every input, which achieves coefficient of determination of 0.999. Hence, we use this linearity to find the exact latent scale at which the d_1 and d_2 logits swap (i.e. intersection between the logits).

4.3. Generalisation to other SAEs

We test for similar behaviours in other SAEs to support generalisation of our results, albeit limited at this experimental scale. We automatically identify special latents by checking each latent's correlation with $\Delta\alpha$ is over threshold $|r| > 0.3$. This threshold was chosen empirically because it reliably selected latents that could be steered to swap . . . TODO

We find that SAEs with strong performance reliably learn special latents (Appendix E, Figure 12) This strongly suggests that the behaviour described so far in this section generalises to various SAE configurations.

We provide a pipeline in Appendix E that identifies special latents in a given SAE and attempts steering on all input bigrams. Since this is computationally expensive, we only test on our selected subset of SAEs (see Appendix B, Table 5). We find that all six sampled SAEs learned at least one special latent ($|r| > 0.3$), and five learned a pair of opposing special latents (one d_1-favouring and one d_2-favouring). In every tested SAE, steering at least one of these special latents successfully swapped the predicted outputs for a substantial portion of the input space, ranging from 26.6% to 72.5% of all inputs. When the output swap failed, the inputs typically either did not activate the latent at all or encountered constraints similar to those detailed in Section 4.2, such as requiring negative activation scaling. This suggests that the relative magnitude-based mechanism, and the emergence of steerable, graded latent activations, generalizes consistently across varying SAE architectures and hyperparameter configurations. Further results are in Appendix E.

5. Discussion

Our SAEs reliably learn latents with graded activations that can be steered to swap model outputs.

However, our SAE experiments demonstrate robust findings across diverse configurations: special latents (characterized by $\Delta\alpha$ correlation) emerge across three distinct SAE architectures (BatchTopK, JumpReLU, Matryoshka) with varying sparsity and dictionary size settings 3.

TODO

6. Conclusion and Future Work

In this paper, we showed that SAEs trained on an activation that uses relative magnitude-based composition reliably learn latents with graded activations with magnitude that correlate with $\Delta\alpha$. We demonstrated that steering these la-

tents predictably changes the transformer's predicted output. We release our trained SAEs on Weights & Biases to support future work and reproducibility of our results. Limitations are listed in Appendix F.

In future work we will attempt to find graded latent activations in real-world pretrained SAEs. The Gemma Scope releases (Lieberum et al., 2024; McDougall et al., 2025) provide a natural test environment, which include JumpReLU and Matryoshka SAEs trained on Gemma 2 and Gemma 3 at multiple widths; the same SAE variants we study here. The specific target would be identifying SAE latents in pretrained models where activation magnitude, rather than feature presence, is causally relevant for downstream behaviour.

In summary, the findings in this paper are intended as a foundation for further research on magnitude-based feature composition mechanisms in LLMs.

Impact Statement

This paper presents work whose goal is to advance our understanding of sparse autoencoders, which are popular tools in interpretability research. If our findings are also present in real-world language models, then this may present significant problems for use of SAEs for interpretability of such systems.

References

Arditi, A., Obeso, O., Syed, A., Paleka, D., Panickssery, N., Gurnee, W., and Nanda, N. Refusal in language models is mediated by a single direction. *Advances in Neural Information Processing Systems*, 37:136037–136083, 2024.

Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., McLean, B., Burke, J. E., Hume, T., Carter, S., Henighan, T., and Olah, C. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. https://transformer-circuits.pub/2023/monosemantic-features/index.html.

Brixi, G., Durrant, M. G., Ku, J., Naghipourfar, M., Poli, M., Sun, G., Brockman, G., Chang, D., Fanton, A., Gonzalez, G. A., King, S. H., Li, D. B., Merchant, A. T., Nguyen, E., Ricci-Tam, C., Romero, D. W., Schmok, J. C., Taghibakhshi, A., Vorontsov, A., Yang, B., Deng, M., Gorton, L., Nguyen, N., Wang, N. K., Pearce, M. T., Simon, E., Adams, E., Amador, Z. J., Ashley, E. A., Baccus, S. A., Dai, H., Dillmann, S., Ermon, S., Guo, D., Herschl, M. H., Ilango, R., Janik, K., Lu, A. X., Mehta, R., Mofrad, M. R. K., Ng, M. Y., Pannu, J., Ré, C., St. John, J., Sullivan, J., Tey, J., Viggiano, B., Zhu, K., Zynda, G., Balsam, D., Collison, P., Costa, A. B., Hernandez-Boussard, T., Ho, E., Liu, M.-Y., McGrath, T., Powell, K., Pinglay, S., Burke, D. P., Goodarzi, H., Hsu, P. D., and Hie, B. L. Genome modelling and design across all domains of life with evo 2. *Nature*, 03 2026. ISSN 1476-4687. doi: 10.1038/s41586-026-10176-5. URL https://doi.org/10.1038/s41586-026-10176-5.

Bussmann, B., Leask, P., and Nanda, N. Batchtopk sparse autoencoders. In *NeurIPS 2024 Workshop on Scientific Methods for Understanding Deep Learning*, 2024. URL https://openreview.net/forum?id=d4dpOCqybL.

Bussmann, B., Nabeshima, N., Karvonen, A., and Nanda, N. Learning multi-level features with matryoshka sparse autoencoders. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=m25T5rAy43.

Chanin, D., Wilken-Smith, J., Dulka, T., Bhatnagar, H., Golechha, S., and Bloom, J. I. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=R73ybUciQF.

Costa, V., Fel, T., Lubana, E. S., Tolooshams, B., and Ba, D. E. Evaluating sparse autoencoders: From shallow design to matching pursuit. In *ICML 2025 Workshop on Methods and Opportunities at Small Scale*, 2025. URL https://openreview.net/forum?id=SLGftRJVUN.

Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.

Farrell, T., Leask, P., and Moubayed, N. A. Order by scale: Relative-magnitude relational composition in attention-only transformers. In *Socially Responsible and Trustworthy Foundation Models at NeurIPS 2025*, 2025. URL https://openreview.net/forum?id=vWRVzNtk7W.

Ferrando, J., Sarti, G., Bisazza, A., and Costa-Jussà, M. R. A primer on the inner workings of transformer-based language models. *arXiv preprint arXiv:2405.00208*, 2024.

Gao, L., la Tour, T. D., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., and Wu, J. Scaling and evaluating sparse autoencoders. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=tcsZt9ZNKD.

Huben, R., Cunningham, H., Smith, L. R., Ewart, A., and Sharkey, L. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=F76bwRSLeK.

Karvonen, A., Rager, C., Lin, J., Tigges, C., Bloom, J. I., Chanin, D., Lau, Y.-T., Farrell, E., McDougall, C. S., Ayonrinde, K., Till, D., Wearden, M., Conmy, A., Marks, S., and Nanda, N. SAEBench: A comprehensive benchmark for sparse autoencoders in language model interpretability. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=qrU3yNfX0d.

Leask, P., Bussmann, B., Pearce, M. T., Bloom, J. I., Tigges, C., Moubayed, N. A., Sharkey, L., and Nanda, N. Sparse autoencoders do not find canonical units of analysis. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL https://openreview.net/forum?id=9ca9eHNrdH.

Leask, P., Nanda, N., and Moubayed, N. A. Inference-time decomposition of activations (ITDA): A scalable approach to interpreting large language models. In *Forty-second International Conference on Machine Learning*, 2025b. URL https://openreview.net/forum?id=4WMZqAoYi4.

Lieberum, T., Rajamanoharan, S., Conmy, A., Smith, L., Sonnerat, N., Varma, V., Kramar, J., Dragan, A., Shah, R., and Nanda, N. Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. In Belinkov, Y., Kim, N., Jumelet, J., Mohebbi, H., Mueller, A., and Chen, H. (eds.), *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 278–300, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.blackboxnlp-1.19. URL https://aclanthology.org/2024.blackboxnlp-1.19/.

Marks, S., Karvonen, A., and Mueller, A. dictionary_learning, 2024. URL https://github.com/saprmarks/dictionary_learning.

Marks, S., Treutlein, J., Bricken, T., Lindsey, J., Marcus, J., Mishra-Sharma, S., Ziegler, D., Ameisen, E., Batson, J., Belonax, T., et al. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025.

McDougall, C., Conmy, A., Kramár, J., Lieberum, T., Rajamanoharan, S., and Nanda, N. Gemma scope 2: Technical paper. Technical report, Google DeepMind, 9 2025. URL https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/gemma-scope-2-helping-the-ai-safety-community-deepen-understanding-of-complex-language-model-behavior/Gemma_Scope_2_Technical_Paper.pdf.

Movva, R., Peng, K., Garg, N., Kleinberg, J., and Pierson, E. Sparse autoencoders for hypothesis generation. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=4R0pugRyN5.

Rajamanoharan, S., Conmy, A., Smith, L., Lieberum, T., Varma, V., Kramar, J., Shah, R., and Nanda, N. Improving sparse decomposition of language model activations with gated sparse autoencoders. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a. URL https://openreview.net/forum?id=zLBlin2zvW.

Rajamanoharan, S., Lieberum, T., Sonnerat, N., Conmy, A., Varma, V., Kramár, J., and Nanda, N. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024b.

Wattenberg, M. and Viégas, F. Relational composition in neural networks: A survey and call to action. In *ICML 2024 Workshop on Mechanistic Interpretability*, 2024. URL https://openreview.net/forum?id=zzCEiUIPk9.

A. Further comparison of SAE Types

Table 2. Comparison of some sparse autoencoder (SAE) variants.

Type	Description
Standard ReLU (Bricken et al., 2023)	Applies an L_1 penalty to hidden activations using a standard ReLU activation function to encourage sparsity.
Gated (Rajamanoharan et al., 2024a)	Decouples feature detection from magnitude estimation using a gated mechanism to prevent the shrinkage issues caused by standard L_1 penalties.
JumpReLU (Rajamanoharan et al., 2024b)	Uses a discontinuous step function at a learned per-feature threshold, approximating an L_0 penalty through straight-through estimation. Notably used in Lieberum et al. (2024).
BatchTopK (Bussmann et al., 2024)	Retains top $n \times k$ activations over a batch of n inputs and sets the rest to zero, where k is a tuned hyperparameter. This forces an average of k latent activations per input, so the actual L_0 sparsity is approximately k. Notably used in Brixi et al. (2026); Karvonen et al. (2025).
Matryoshka (Bussmann et al., 2025)	Trains nested sub-dictionaries of increasing size that share a single encoder/decoder, producing multiple resolutions of feature granularity in one run. Notably used in McDougall et al. (2025).

An SAE consists of a single hidden layer trained to reconstruct model activations subject to a sparsity penalty on the latent activations (typically L_1 or top-k regularisation) (Huben et al., 2024; Bricken et al., 2023). Many SAE variants exist in the literature, some of which are shown in Table A.

We use 3 types of SAEs in this paper. JumpReLU SAEs (Rajamanoharan et al., 2024b) use a discontinuous step function at a learned threshold and optimise an L_0 penalty. BatchTopK SAEs (Bussmann et al., 2024) keep the top $n \times k$ activations over a batch of n inputs and sets the rest to zero. This forces an average of k latent activations per input, so the actual L_0 sparsity is approximately k. Matryoshka SAEs (Bussmann et al., 2025) train nested sub-dictionaries of increasing size under a shared encoder and decoder, producing representations at multiple levels of granularity from a single training run.

B. Experimental Setup

Table 3. SAE hyperparameter sweep configurations. Braces denote swept values; fixed values are listed directly. Fixed parameters across all runs: 50,000 training steps, 1,000 warmup steps, 4,096 batch size. Total run counts are the product of all swept values and seeds. Note that $n_{\text{groups}} = 1$ in a Matryoshka SAE reduces it to a standard BatchTopK SAE, so this serves as an internal consistency check.

Parameter	**Architecture**	**Values**
d_{SAE}	All	$\{128, 192, 256, 320\}$
k / Target L_0	All	$\{3, 4, 5\}$
	BatchTopK	$+\{1, 2\}$
	JumpReLU	$+\{2\}$
Learning rate	BatchTopK, Matryoshka	3×10^{-4}
	JumpReLU	$\{7\times10^{-5}, 3\times10^{-4}\}$
Sparsity penalty	JumpReLU	$\{0.1, 0.5, 1.0, 5.0\}$
n_{groups}	Matryoshka	$\{1, 2, 4\}$
Architecture	**Seeds**	**Total runs**
BatchTopK	15	450
JumpReLU	5	640
Matryoshka	15	540

Table 4. SAE sweep results aggregated over multiple seeds (mean ± std). L_0: mean active features per token. d_{SAE}: dictionary size. **LR** (loss recovered): fraction of model loss recovered by the SAE reconstruction (↑ better). **Dead %**: percentage of dictionary features with zero activation across the evaluation set (↓ better). **Bold**: best within each L_0 block; **underlined**: best overall.

L_0	d_{SAE}	Runs	LR (↑)	Dead % (↓)
1	128	18	0.9393 ± 0.0022	$\mathbf{12.8 \pm 2.8}$
1	192	15	$\mathbf{0.9410 \pm 0.0014}$	43.9 ± 1.9
1	256	15	0.9406 ± 0.0011	53.9 ± 2.0
1	320	15	0.9397 ± 0.0015	61.9 ± 1.3
2	128	16	$\mathbf{0.9822 \pm 0.0011}$	$\mathbf{17.7 \pm 1.8}$
2	192	15	0.9808 ± 0.0031	42.6 ± 1.4
2	256	15	0.9811 ± 0.0022	54.4 ± 1.5
2	320	15	0.9802 ± 0.0025	61.6 ± 1.8
3	128	19	0.9948 ± 0.0155	$\mathbf{14.1 \pm 2.9}$
3	192	14	$\mathbf{0.9996 \pm 0.0001}$	34.8 ± 4.8
3	256	12	$\mathbf{0.9996 \pm 0.0001}$	48.2 ± 7.1
3	320	13	$\mathbf{0.9996 \pm 0.0001}$	54.1 ± 6.9
4	128	17	$\mathbf{0.9996 \pm 0.0000}$	$\mathbf{1.1 \pm 0.9}$
4	192	12	$\mathbf{0.9996 \pm 0.0000}$	5.1 ± 2.0
4	256	15	$\mathbf{0.9996 \pm 0.0001}$	17.9 ± 15.8
4	320	16	$\mathbf{0.9996 \pm 0.0001}$	19.4 ± 12.6
5	128	17	0.9996 ± 0.0000	$\underline{\mathbf{0.3 \pm 0.4}}$
5	192	19	0.9996 ± 0.0000	5.2 ± 8.3
5	256	17	0.9996 ± 0.0000	7.2 ± 3.3
5	320	15	$\underline{\mathbf{0.9997 \pm 0.0000}}$	11.8 ± 3.2

C. Further SAE Latent Analysis

Table 5. Comparison of cherry-picked SAE architecture types across configuration and performance metrics. These SAEs are used to support generalisation claims when a large testing sample is not computationally feasible. Dashes indicate parameters not applicable to a given architecture. L_0 (actual) is the mean number of active latents measured at eval time. **Bold**: best value per performance metric.

SAE Type	Configuration					Performance			
	d_{SAE}	LR	k / Target L_0	Sp. Pen.	n_{groups}	CE Increase ↓	Expl. Var. ↑	Dead (%) ↓	L_0 (actual)
BatchTopK	192	3×10^{-4}	3	—	—	0.0257	0.9929	26.04	3.36
	128	3×10^{-4}	5	—	—	0.0292	0.9939	**0.78**	4.90
JumpReLU	256	7×10^{-5}	5	5	—	**0.0141**	**0.9965**	50.78	4.03
	128	7×10^{-5}	5	5	—	0.0176	0.9963	17.97	**3.24**
Matryoshka	256	3×10^{-4}	3	—	2	0.0262	0.9928	39.06	3.32
	192	3×10^{-4}	4	—	4	0.0325	0.9927	12.50	3.98

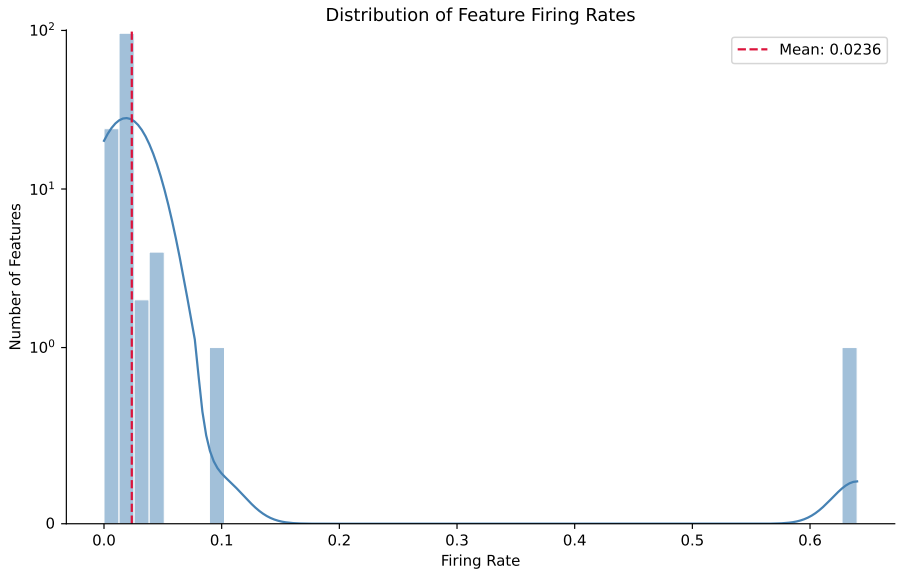

Figure 3. This histogram shows the distribution of firing rates for the SAE checkpoint's latents. The y-axis uses a symmetric logarithmic scale with a linear scale between $[-1, 1]$. A given latent's firing rate is the proportion of inputs which cause that latent to activate. The special latents are the only ones with a firing rate greater than one standard deviation from the mean (mean: 0.0236, standard deviation: 0.0559): latent 11 has rate 0.6400, latent 56 has rate 0.0991.

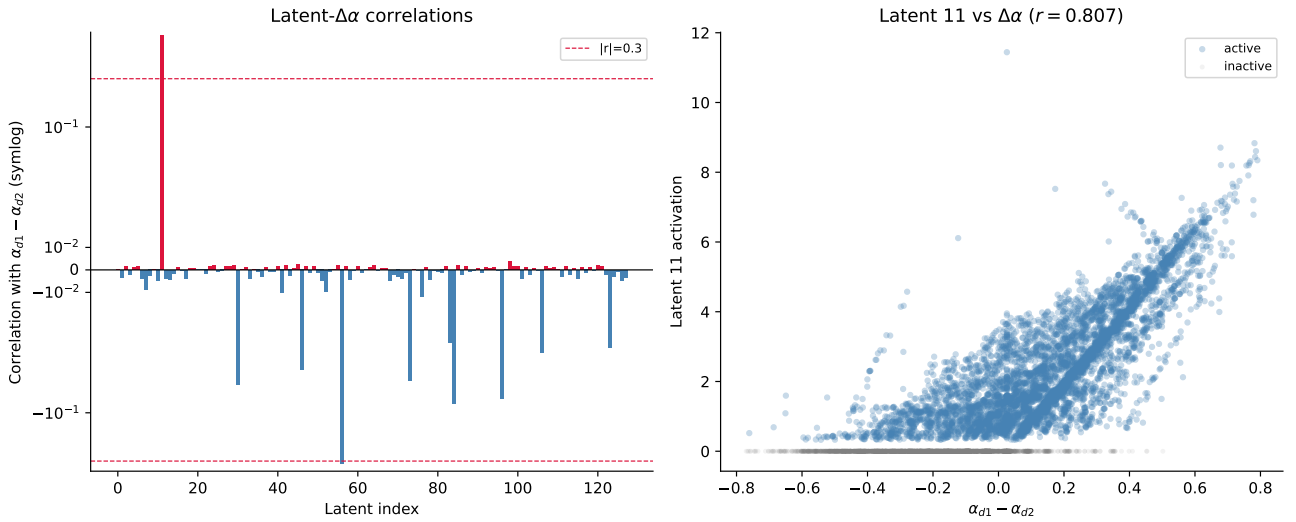

Figure 4. **Left**: This plot shows the (Pearson) correlation of each latent with $\Delta\alpha = \alpha_{s\to d_1} - \alpha_{s\to d_2}$, which is intuitively the emphasis given to d_1 over d_2 by SEP in the transformer model's first layer's attention pattern. The y-axis uses a symmetric logarithmic scale with a linear scale between $[-0.05, 0.05]$. d_1-favouring latents are highlighted red and d_2-favouring latents are highlighted blue. The top three strongest correlations are: latent 11 with $r = 0.807$, latent 56 with $r = -0.3213$, and latent 96 with $r = 0.0036$. **Right**: This plot shows the correlation of latent 11 with $\Delta\alpha$ in more detail, as this latent has the strongest correlation. All 10,000 possible input bigrams are plotted, and highlighted blue if they activate latent 11 and grey otherwise. We see that the latent activates with greater magnitude for larger $\Delta\alpha$.

D. Further Latent Steering Examples

Figure 5 shows an example where swapping o_1 requires negative scaling ($p < 0$). The logit for d_2 at o_1 only overtakes d_1 when the latent activation is suppressed below zero, which is outside the valid activation range of a BatchTopK SAE.

Figure 6 shows an example where the swap ranges for o_1 and o_2 are mutually exclusive: there exists a positive p that swaps each position individually, but no single p swaps both simultaneously.

Figures 7 and 8 show examples where no swap occurs across the full observed range $p \in [0, 10]$. In Figure 7, the logit for d_1 remains dominant at o_1 for all p; Figure 8 shows the analogous failure at o_2.

In limited experiments with inputs that fail for the first three causes, it is sometimes possible to scale a co-activating (non-special) latent to successfully swap outputs. Figures 9, 10 and 11 show an example where input $(41, 49)$ activates latents 11 (magnitude 3.1), 0 (magnitude 3.1) and 55 (magnitude 2.5). The swap fails with latent 11 because there are no positive overlapping scale ranges; however, scaling latent 55 by $p \in [1.1, 1.15]$ successfully produces a swap. This does not necessarily indicate that latent 55 is graded; rather, scaling it alters the logit distribution, which opens a valid swap zone for latent 11. Similarly, running the steering pipeline on latent 55 alone swaps only 4 inputs, all of which co-activate latent 11, suggesting a similar mechanism. We note this preliminary experiment is insufficient for statistical significance, but it suggests that scaling multiple latents together may enable more output swaps than steering special latents alone.

We do not attempt steering on all latents due to computational constraints, but we attempt it on five randomly selected latents (0, 55, 63, 82, 117) and latent 96 (non-special latent with highest $\Delta\alpha$ correlation, $r = 0.0036$) and latent 5 (which is shown in Figure 2). Results are shown in Table 6. We find that every single successful swap using a non-special latent has a co-activating special latent, which suggests that these non-special latents are unlikely to be graded, otherwise they could singlehandedly swap more outputs, and instead alter the logit distribution such that the graded special latents can successfully swap outputs.

D.1. Steering Across Multiple SAEs

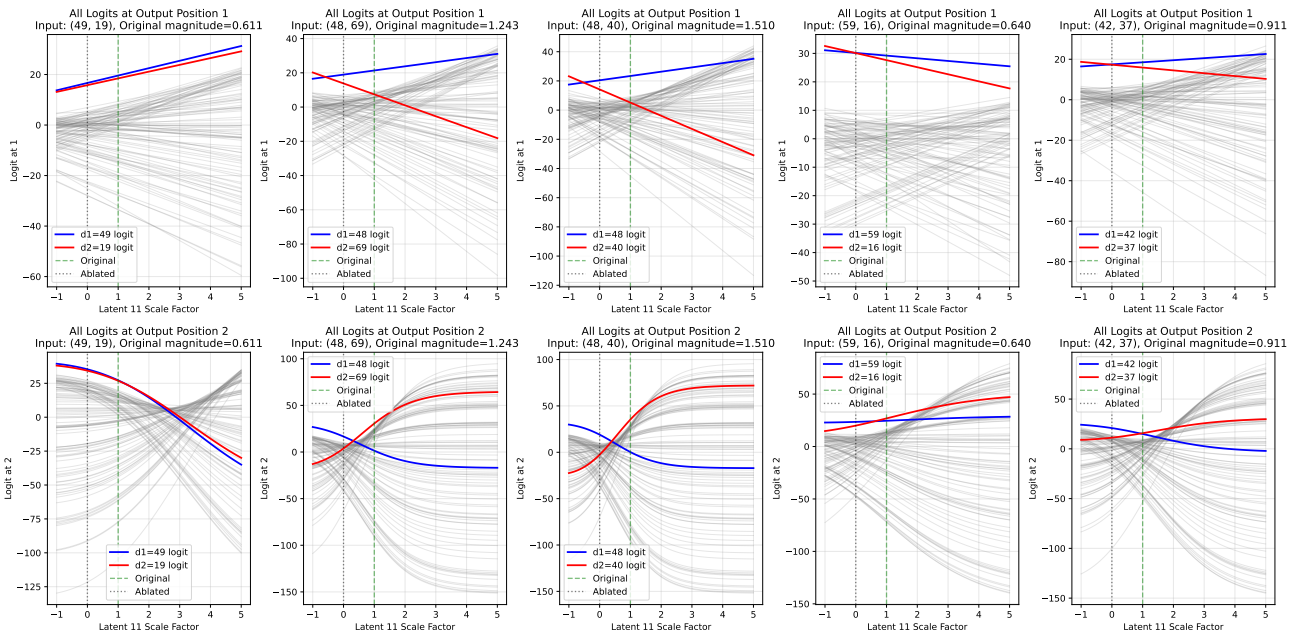

Figure 5. For 9.2% of inputs, latent 11 (in the BatchTopK SAE from Section 3) must be negatively scaled to swap the predicted symbol at one of the output positions.

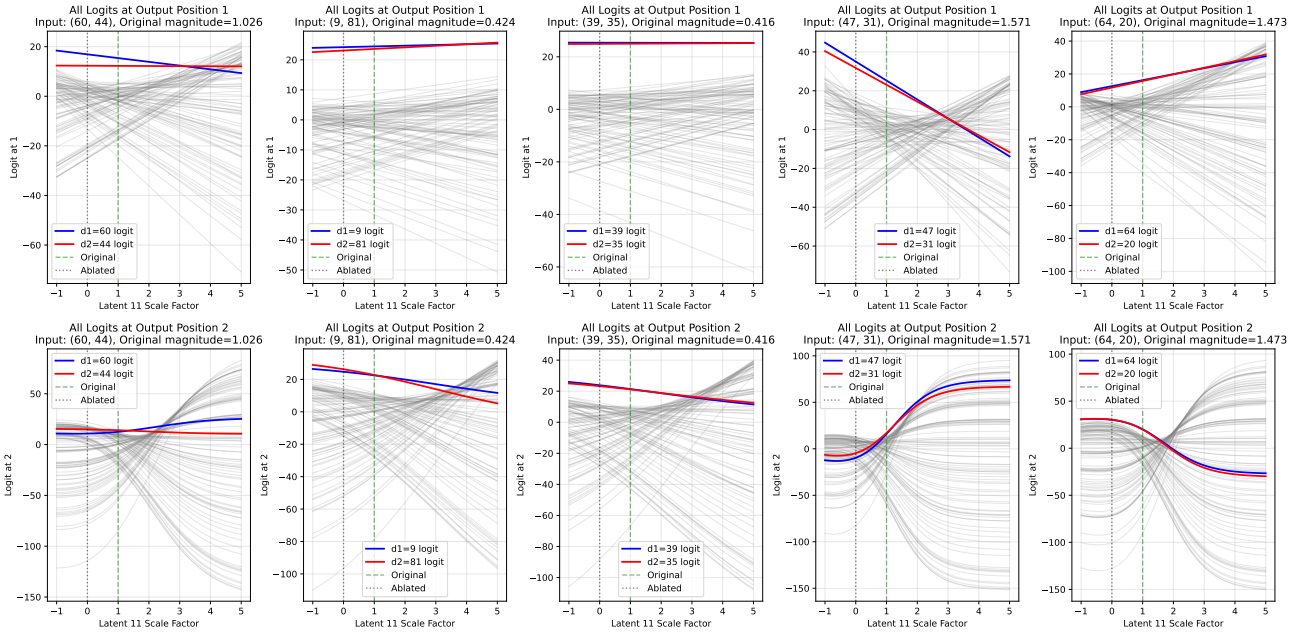

Figure 6. For 2.3% of inputs, it is possible to scale latent 11 (in the BatchTopK SAE from Section 3) to swap the symbol at each output position individually but not simultaneously.

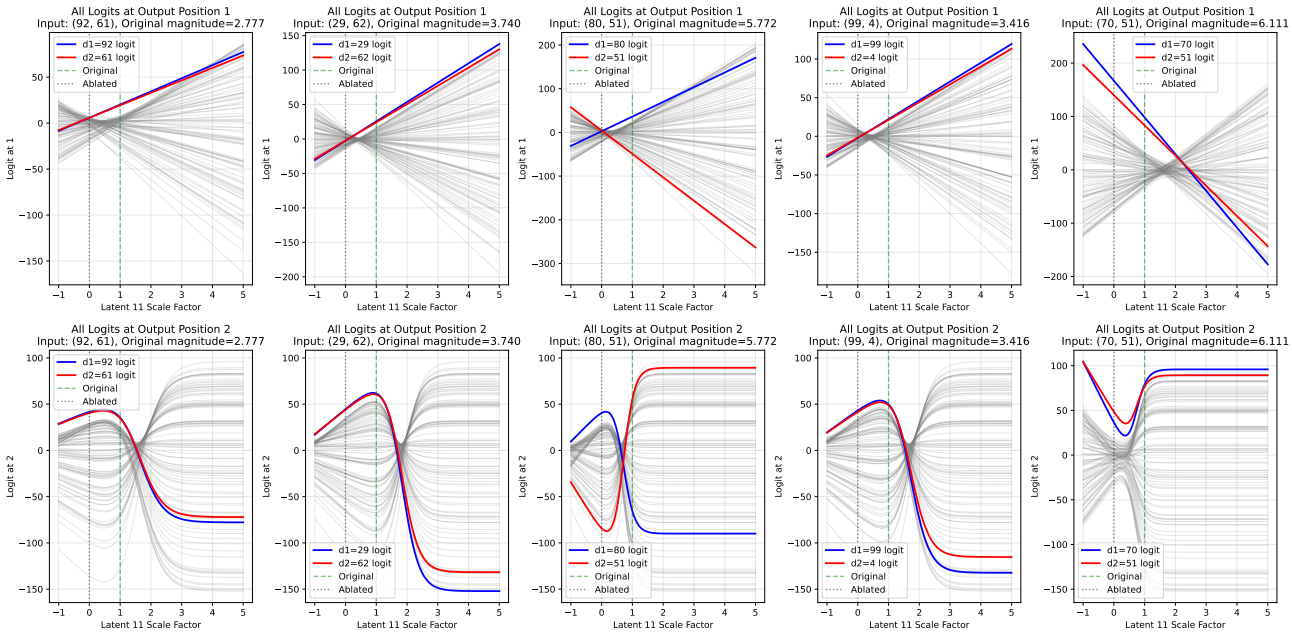

Figure 7. For 1.6% of inputs, the logit for the symbol at d_2 is always dominant at o_2 in the observed latent scale range (in the BatchTopK SAE from Section 3), so there is no scale where the the symbol at d_1 is predicted instead.

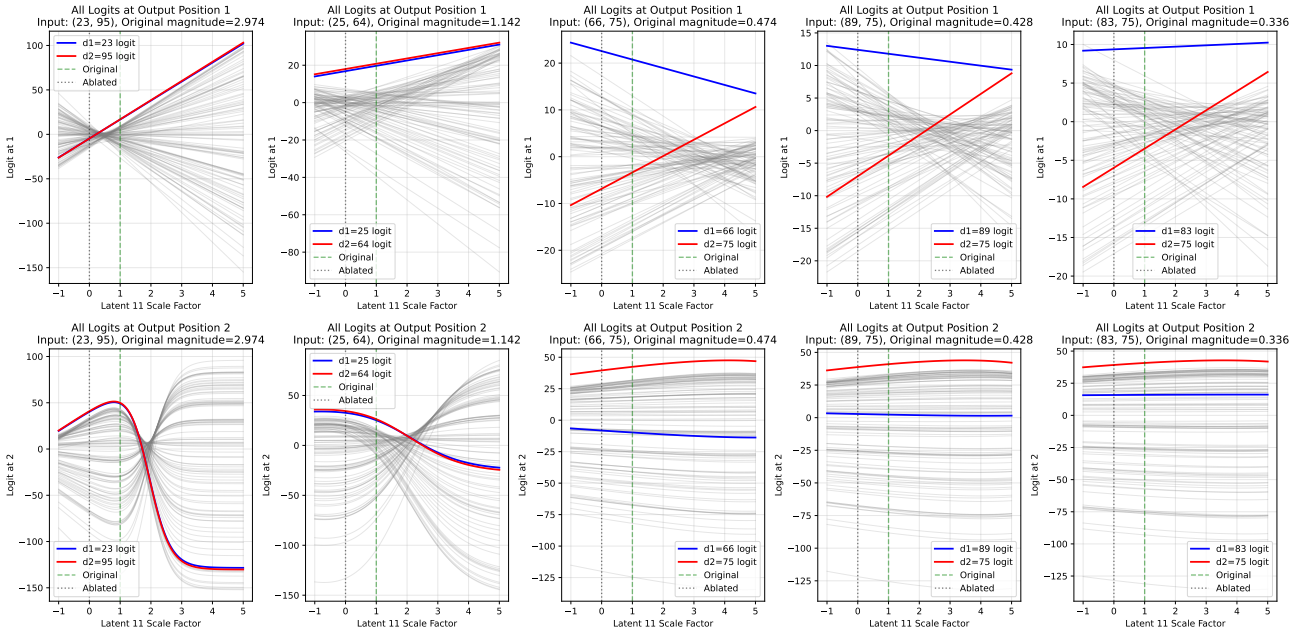

Figure 8. For 0.2% of inputs, the logit for the symbol at d_1 is always dominant at o_1 in the observed latent scale range (in the BatchTopK SAE from Section 3), so there is no scale where the the symbol at d_2 is predicted instead.

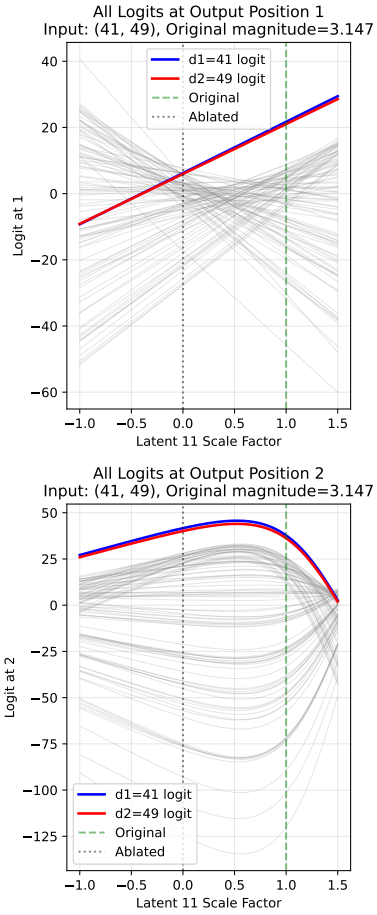

Figure 9. In the BatchTopK SAE from Section 3, latent 11 cannot be scaled to swap input bigram $(41, 49)$ because the logit for 49 is never argmax in position o_1, as shown by the top plot.

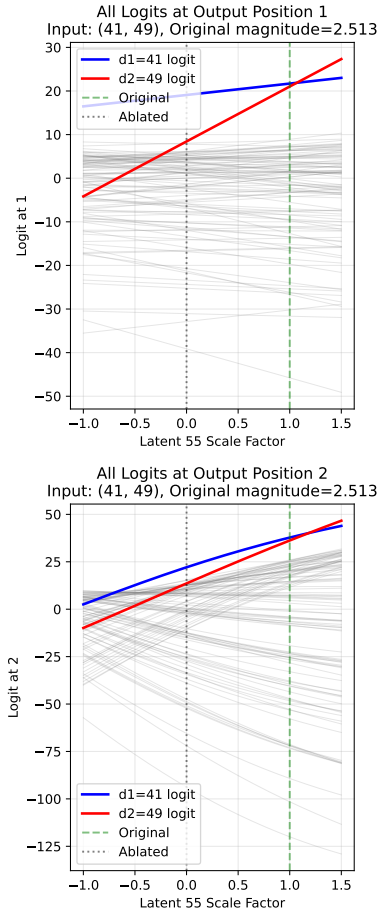

Figure 10. However, in the same SAE, latent 55 co-activates with latent 11 for input $(41, 49)$ and can be scaled by factor $p \in [1.07, 1.18]$ to cause the model to output $(49, 41)$ instead.

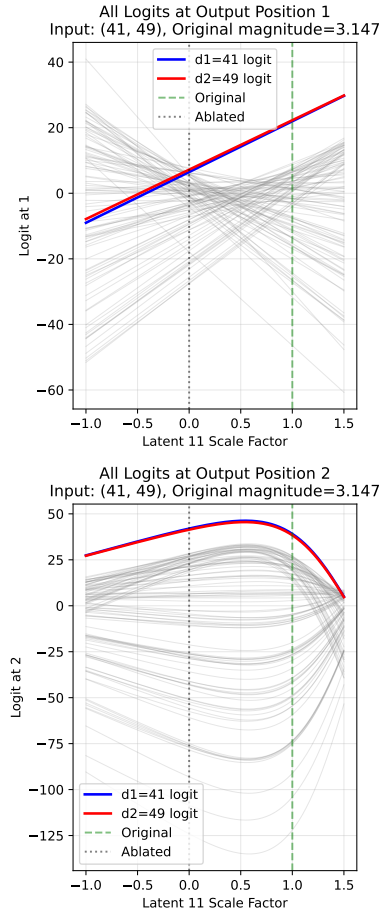

Figure 11. Moreover, in the same SAE, if latent 55's activation magnitude is scaled by a factor of 1.1, then we can now scale latent 11 by factor $p \in [0.03, 1.45]$ to swap the outputs, which was not possible without scaling latent 55. This suggests that some graded activation demonstrations are only possible when multiple active latents are scaled simultaneously.

Table 6. We perform steering experiments on several chosen and randomly sampled SAE latents to demonstrate how steering with special latents (**bold**) compares to non-special latents. We record how many outputs are successfully swapped by steering only that latent, how many do not activate the latent, and how many activate the latent but cannot be swapped.

Latent	Swapped	No Activation	Active & No Swap
0	6 (0.1%)	9786 (97.9%)	208 (2.1%)
5	23 (0.2%)	9594 (95.9%)	383 (3.8%)
11	**4976 (49.8%)**	**3600 (36.0%)**	**1424 (14.2%)**
55	4 (0.0%)	9787 (97.9%)	209 (2.1%)
56	**196 (2.0%)**	**9009 (90.1%)**	**795 (7.9%)**
63	4 (0.0%)	9801 (98.0%)	195 (2.0%)
82	4 (0.0%)	9787 (97.9%)	209 (2.1%)
117	37 (0.4%)	9782 (97.8%)	181 (1.8%)

Table 7. Generalisation of special latent steering to other SAE configurations from Table 5. For each identified special latent ($|r| > 0.3$), we report its bias, Pearson correlation r with $\Delta\alpha$, and the percentage of inputs where the latent was successfully steered to swap outputs (Success), did not activate (No Act.), or activated but could not be swapped (Active & No Swap).

SAE Type	d_{SAE}	L_0	Latent	Type	r	Steering Outcomes (%)		
						Success	No Act.	Active & No Swap
BatchTopK (Studied)	128	3.00	11	d_1-favouring	$+0.807$	49.8%	36.0%	14.2%
			56	d_2-favouring	-0.321	2.0%	90.1%	7.9%
BatchTopK	192	3.36	47	d_2-favouring	-0.836	52.9%	34.4%	12.7%
			116	d_1-favouring	$+0.365$	0.5%	91.7%	7.8%
	128	4.90	68	d_1-favouring	$+0.845$	32.9%	44.7%	22.4%
			64	d_2-favouring	-0.730	6.0%	62.0%	32.0%
JumpReLU	256	4.03	195	d_1-favouring	$+0.849$	38.8%	39.8%	21.4%
			179	d_2-favouring	-0.721	5.8%	60.2%	34.0%
	128	3.24	105	d_2-favouring	-0.816	28.6%	37.8%	33.6%
			4	d_1-favouring	$+0.756$	6.9%	62.3%	30.8%
Matryoshka	256	3.32	48	d_2-favouring	-0.796	26.6%	50.6%	22.8%
			62	d_1-favouring	$+0.591$	6.0%	71.2%	22.8%
	192	3.98	30	d_1-favouring	$+0.866$	72.5%	16.5%	11.0%

E. Further Generalisation Experiment Details

To test whether special latents can be steered to swap outputs in other SAEs, we provide the following pipeline that works given any SAE, which is available in our public codebase:

1. Store the SAE activations for all input bigrams by running a single forward pass over the full dataset and recording each SAE latent's activation magnitude.
2. Identify special latents using $\Delta\alpha$ correlation $|r| > 0.3$ as a heuristic, as described previously.
3. For each identified special latent, run a batched steering grid search over all input bigrams. For each bigram and each scale factor $p \in [0, 10]$ with step size 0.05 (201 values), record the logits at output positions o_1 and o_2 under the steered reconstruction $\hat{\boldsymbol{r}}_s^{L_1}$. Bigrams for which the latent does not activate ($m = 0$) are skipped immediately since scaling has no effect.
4. Find crossover scales for each bigram:
    - For o_1: since o_1 logits are empirically linear in p, fit lines to $\ell_{d_1}(p)$ and $\ell_{d_2}(p)$ and compute the analytical

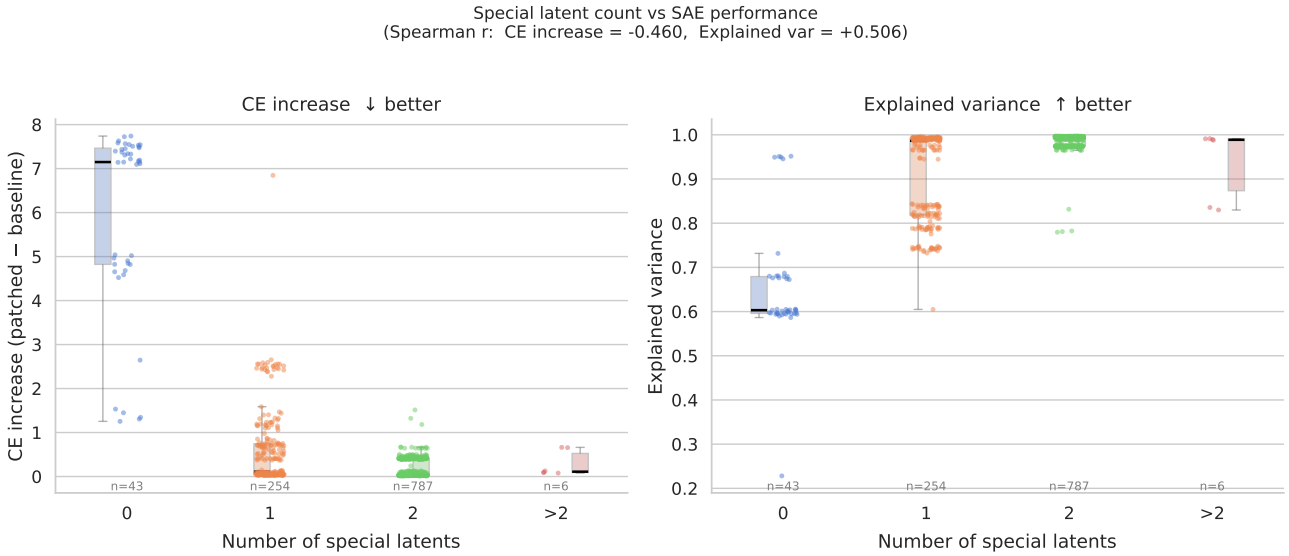

Figure 12. These strip plots show how the number of special latents (identified by the latent's activation's correlation with $\Delta\alpha$ using threshold $|r| > 0.3$, as in Section 4.1) differs across SAEs with different performance. It shows that SAEs with strong performance, measured by explained variance and patched cross-entropy loss, tend to have 2 special latents (spearman $r = -0.460$ for patched cross-entropy loss and $r = 0.506$ for explained variance).

intersection. Inputs for which either logit series fails an $R^2 \geq 0.999$ linearity check, or for which the intersection falls on a negative scale, are flagged and excluded.

- For o_2: detect sign changes in $\ell_{d_1}(p) - \ell_{d_2}(p)$ on the coarse $[0, 10]$ grid, then refine each crossing to three decimal places using bisection search. All bisection tasks across the batch are executed in parallel to avoid sequential per-input forward passes.

5. Determine a valid swap zone $[p_{\text{lo}}, p_{\text{hi}}]$ per bigram by intersecting the o_1 and o_2 crossover constraints with the scale ranges over which $\text{argmax}(o_1) = d_2$ and $\text{argmax}(o_2) = d_1$ hold simultaneously. If no contiguous window satisfying both conditions exists, the bigram is recorded as a failure with a specific reason (see Section 4.2).

6. Verify each swap zone by running the model at the midpoint $p^* = (p_{\text{lo}} + p_{\text{hi}})/2$ and confirming that the model output is (d_2, d_1).

7. Report the fraction of bigrams for which a valid swap zone was found and the swap verified, along with a breakdown of failure reasons for the remainder.

Steps 3-5 of the pipeline are very computationally expensive, so we test generalisation using a varied sample of six high performing SAEs (Table 5) rather than running the pipeline on all trained SAEs (as in Figure 12). The results are shown in Table 7.

In future work, this study should be extended to further configurations for robust statistical significance.

F. Limitations

A. **Minimal toy model.** The transformer is trained on a single narrow task and omits components standard in pretrained LLMs (MLPs, LayerNorm, multi-head attention, value and output projections), with a vocabulary of only 100 symbols. Whether relative magnitude-based composition appears in less constrained models remains an open question.

B. **Single-latent steering.** The steering pipeline handles one special latent at a time. The co-activation experiments in Appendix D suggest that simultaneous multi-latent steering could recover additional swaps.

C. **SAE selection bias.** Our primary SAE was selected for having few dead latents, biasing toward high-quality SAEs. A fully representative study would require analysis across a random sample of all trained SAEs, not just well-performing ones.

D. **Limited SAE generalisation.** Special latents emerge across BatchTopK, JumpReLU, and Matryoshka architectures, but generalisation was tested on only six cherry-picked SAEs over at most 15 seeds. Broader statistical significance testing across more configurations is needed.